Surface Tension: An Elicitation Gap, Not a Cost, Bounds Bare-Prompt Compliance with a Code-Style Constraint

Julian Quick

Abstract

We ask whether small-data fine-tuning can convert a prompt-elicited code-style behavior — writing Python without `for`/`while` loops and without recursion — into a *default* for a 31B instruction-tuned code model (Gemma 4 31B-it) on competition-style problems (LCB-medium). We find that the bare-prompt non-compliance is, to a first approximation, an *elicitation gap* rather than a cost trade-off: the constraint costs ≈ 8 pass-rate points on average (not the ≈ 34 a common pilot metric suggests, which is an artifact of an unreliable compliance column), and on $\approx 76\%$ of always-loop-bare problems the model produces a compliant-and-passing solution *when instructed* at $n = 3$ — by rewriting the loop as a comprehension or `reduce`. We characterize the bare-prompt distribution as per-problem near-deterministic on the loop axis (the $[0.3, 0.7]$ "discriminating zone" is essentially empty across the base model and several fine-tunes), which is also why GRPO from base fails: every rollout group is uniform, advantage is zero. A properly-tuned LoRA SFT installs a partial loop-avoidance reflex — generations contain `itertools` imports and "avoid loops" comments. The strength of the reflex depends sharply on the deploy checkpoint, and on *both* a val set used for early-stopping and a clean set used for neither training nor selection the over-fit endpoint (`-final`, epoch 20) substantially outperforms the val-NLL-minimum (`-bestval`, epoch ≈ 4): 0.51 vs 0.37 on the val set, 0.26 vs 0.09 on the clean set. Compliance $\wedge$ pass follows the same pattern. Per-problem flip rates are 75% vs 67% on the val set, 50% vs 19% on the clean set (vs 0% for base). We discuss two methodological lessons: a small val set reused for early-stopping and reporting inflates apparent generalization $\approx 4\times$ at the val-min checkpoint; and val-NLL minimum is itself a poor selector for AST compliance — the same training continued through nominal overfitting is materially better on both the set it was selected on and a clean third set.

1 Introduction

A 31B instruction-tuned code model can comply with a stylistic AST constraint — here, "no `for`/`while` statements and no recursion" — when the rule is in the prompt, but rarely complies by default. Closing that prompt–default gap by fine-tuning is the kind of internalization task active in the capability-elicitation literature (Greenblatt et al., 2024; Ryd et al., 2026). We probe it on Gemma 4 31B-it against LiveCodeBench-medium (Jain et al., 2025), with four contributions:

1. **The gap is an elicitation gap, not a cost.** The constraint is mostly free — ≈ 8 pass-rate points across the LCB pilot, not the ≈ 34 a common pilot metric reports (the metric was an artifact of treating empty AST trees as compliant). Of the $\approx 93\%$ of LCB problems where the base model loops by default, $\approx 76\%$ yield a compliant-and-passing solution when the rule is in the prompt — usually by rewriting the loop as a list comprehension or `reduce` (§3).
2. **The bare-prompt distribution is per-problem near-deterministic.** At $T = 0.7$, 8/8 LCB problems sampled at $n \approx 14$ each fall within 0.1 of compliance rates 0 or 1. On

a 200-problem expanded set, ≈ 1% of problems are strictly between. This bimodality survives across the base model, an under-trained SFT, an over-trained SFT, and a KTO run (§4). It is also why GRPO from base fails: rollout groups are uniformly all-0 or all-1, advantage is zero (§6).

3. **Small-data SFT installs a partial loop-avoidance *reflex* whose strength depends sharply on the deploy checkpoint.** A LoRA fine-tune on 91 (bare-prompt, compliant-completion) demonstrations covering 28 LCB problems yields, on a 17-problem clean set used for neither training nor selection, ≈ 0.09 compliance at the val-NLL-minimum checkpoint (`-bestval`) but ≈ 0.26 at the over-fit endpoint (`-final`); 0.04 vs 0.16 on compliance ∧ pass (§5). On the val set itself, `-bestval` reaches 0.37 (inflated by selection on that set) and `-final` reaches 0.51 — so the val-NLL-minimum deploy is materially worse on *both* the val set it was selected on and a clean third set. Per-problem flip rates are 67% vs 75% on the val set, 19% vs 50% on the clean set (0% for base).
4. **Methodological lesson.** The 12 validation problems were used both to pick the deploy checkpoint (by NLL) and to report compliance. Reporting them as "held out" inflated the apparent generalization roughly 4×. Sampling 5 further problems (the clean set) was sufficient to flip the apparent conclusion from "SFT generalizes" to "small per-problem effect." We also report a separate trap: relying on an eval harness's "compliant" column that misclassified empty AST trees as compliant inflated the pilot's apparent compliance *cost* by 4×. We recommend that any compliance figure derived from saved sources be re-checked end-to-end, and that any generalization figure not reuse problems used for checkpoint selection.

2 Setup

Model and constraint. We use Gemma 4 31B-it (instruction-tuned, mixture-of-experts code model) in 4-bit quantization with QLoRA-style fine-tuning (Dettmers et al., 2023; Hu et al., 2022). The constraint `no_loops_no_recursion` is implemented as an AST predicate: `check_no_loops` forbids `for` and `while` *statements* (list/generator comprehensions, `functools.reduce`, and `itertools` builtins are permitted); `check_no_recursion` forbids self-calls within a function's own body (mutual recursion is conservatively under-flagged). Compliance is the conjunction. Constraint cost is measured against unit tests from the problem source.

Data and evaluation. We work with the LCB-medium subset from LiveCodeBench (57 problems). The SFT corpus is 146 (bare-prompt, compliant-completion) pairs over 40 LCB problems, generated by prompting the base model *with* the constraint and keeping only the AST-compliant completions; we verified the corpus is 100% compliant. We split: 91 pairs / 28 problems for training (**train**), 55 pairs / 12 problems for validation (**val**); a further 17 LCB problems were never used for training or selection (**clean**). All evals are bare-prompt at $T = 0.7$, $n = 8$ unless noted, with AST compliance *re-derived from saved `.py` source files*: the eval harness's CSV `compliant` column is unreliable because `check_no_loops` on an empty AST returns `True`, so `code_extracted = 0` rows are misclassified.

Training. LoRA on stripped attention/MLP projections (Gemma 4's `Gemma4ClippableLinear` wrappers must be removed before LoRA attach: without stripping, the LoRA delta receives zero gradient because the wrapper bypasses its inner `nn.Linear`, and "trained" adapters are the identity — a silent no-op that affected several prior runs in this project). We sweep $r \in \{8, 32, 128\}$ at $\alpha = 2r$, lr $= 10^{-4}$, linear-decay-to-10% schedule, 20 epochs (≈ 228 optimizer steps with effective batch 8 on 91 training pairs); we evaluate held-out NLL every 24 steps and save the validation-NLL-minimum checkpoint as `-bestval`. Other reported runs: lr $= 10^{-5}$ for

3 epochs (under-trained "v9"); lr $\in \{10^{-4}, 3\times10^{-4}\}$ for 8/20 epochs ("Phase A"); a KTO recipe (Ethayarajh et al., 2024); and a GRPO recipe (Shao et al., 2024) from the base model.

3 The Elicitation Gap

We re-derive the pilot from the saved source files (Table 1, Figure 1). On 54 LCB problems where both bare and constrained-prompt evals exist at $n = 3$, the base model has mean pass rate 0.87 unconstrained and 0.79 when told "no loops, no recursion"; on 47 of the 49 problems passable bare, the constraint is free (Δ pass ≈ 0). Of 50 always-loop-bare problems with constrained-condition data, 76% (38/50) produce *at least one compliant-and-passing* solution when the rule is in the prompt at $n = 3$; the remaining 24% split into "compliant but fails" (6%, 3/50) and "no compliant solution found" (18%, 9/50). The headline "≈ 34-point cost" that appears in earlier drafts of this project is an artifact of the unreliable `compliant` column (§2): the column treats empty AST trees as compliant, inflating both the base's apparent compliance rate and the apparent cost of the constraint.

LCB problems with both bare and constrained data (base, $n = 3$)	54
Mean pass rate, bare prompt	0.87
Mean pass rate, constrained prompt (`no_loops_no_recursion`)	0.79
Δ pass rate (constraint cost)	**−0.08**
Bare-prompt passable problems where constrained also passes	47/49
Bare-prompt always-loop problems (compliance = 0 at $n = 3$)	51/54
of which constrained ⇒ compliant-and-passing	**38/50** (**76**%)
constrained ⇒ compliant but failing tests	3/50 (6%)
constrained ⇒ no compliant solution found	9/50 (18%)

Table 1: Pilot, re-derived. The constraint is mostly free; bare-prompt non-compliance is largely an elicitation phenomenon. Decomposition on the 50 always-loop-bare problems for which constrained-condition data exists. $n = 3$ provides a lower bound on the recoverable fraction.

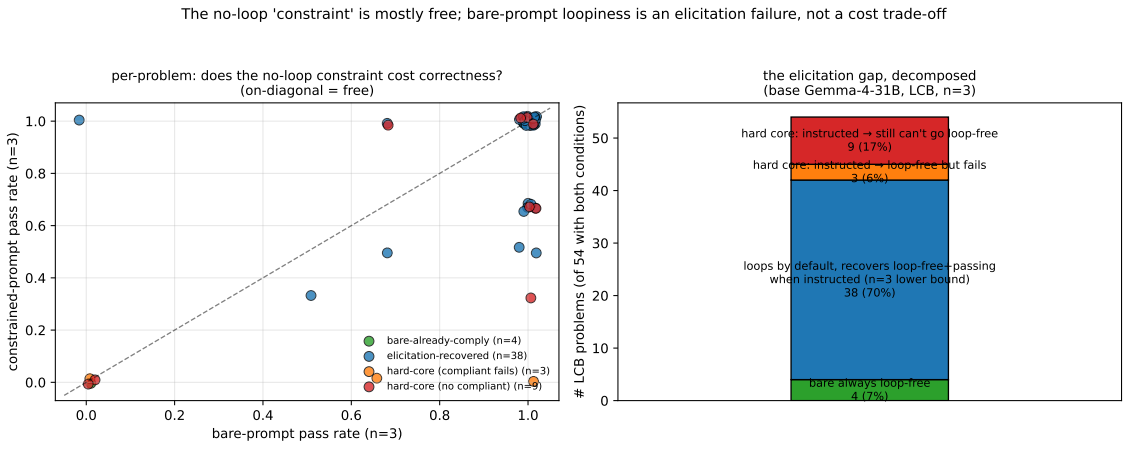

Figure 1: **Left:** per-problem bare-prompt vs constrained-prompt pass rate; on-diagonal = the constraint is free. **Right:** on 54 LCB problems, 7% already bare-compliant, 70% become compliant-and-passing when instructed, 6% become "compliant but fails tests," and 17% remain non-compliant even when told the rule (at $n = 3$).

4 Bimodal Determinism

At $T = 0.7$ the bare-prompt loop decision is per-problem near-deterministic. On 9 LCB problems sampled at $n \in \{6, \ldots, 20\}$, 8/8 problems with data fall within 0.1 of compliance rate 0 or 1. On a 200-problem expanded set (HumanEval + sanitized MBPP (Austin et al., 2021; Chen et al., 2021), base model), $\approx 52\%$ are always-loop, $\approx 46\%$ are always-loop-free, and only $\approx 1\%$ lie strictly in between. We re-measured the same partition under the underfit v9 SFT, the over-fit Phase A 20-epoch checkpoint, and the KTO checkpoint: each is at most $\approx 1\%$ in the $[0.3, 0.7]$ zone. The discriminating zone is essentially empty across these models.

This is the binding precondition for verifiable-reward RL. With G rollouts per prompt at $T = 0.7$, a per-problem bimodal distribution means the rollout group is uniformly all-0 or all-1 on $\approx 99\%$ of prompts: GRPO's group-relative advantage is zero, no gradient. We confirmed this on a GRPO run from base: 19/19 initial steps had advantage variance ≤ 0 (a single step had nonzero loss from an unclipped numerical edge), the mini-eval pass rate fell to 0, and the abort fired at step 20. We do not view this as evidence that the verifiable-reward objective is wrong; it is evidence that the precondition — within-group reward variance — is not satisfied by this combination of base model, constraint, and temperature.

5 SFT: Validation vs Clean Set

We trained three LoRA ranks ($r \in \{8, 32, 128\}$, $\alpha = 2r$, lr $= 10^{-4}$, linear-decay) for 20 epochs and tracked teacher-forced NLL on the held-out 12-problem val set every 24 steps. The three runs are essentially indistinguishable on either curve (Figure 2): train NLL collapses toward 0, val NLL bottoms at epoch ≈ 4 then climbs — the familiar U-shape. Rank is not the bottleneck. The natural deploy checkpoint is the validation-NLL minimum; we save it as `-bestval` (epoch ≈ 4, step 48, val NLL ≈ 0.33 for $r = 8$ and ≈ 0.35 for $r = 32$) and the over-fit endpoint as `-final`.

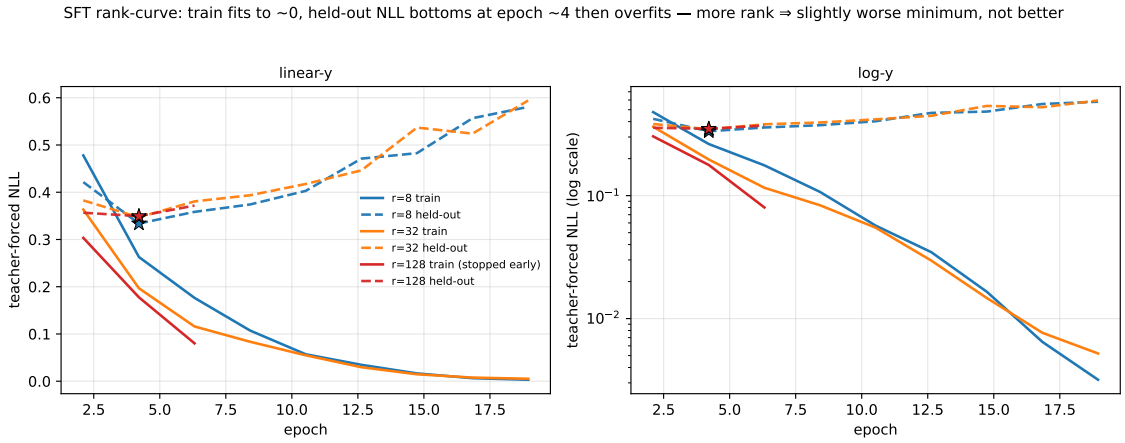

Figure 2: Training-NLL (solid) and held-out-val-NLL (dashed) per epoch for $r \in \{8, 32, 128\}$. Stars mark the val-NLL minimum (the `-bestval` checkpoint). $r = 128$ was halted early; $r = 8$ and $r = 32$ ran the full 20 epochs.

We sample-evaluated the $r = 32$ `-bestval` and `-final` checkpoints bare-prompt at $n = 8$ on the 12 val problems and the 17 clean problems (Table 2). The `-bestval` val-set headline is $34/93 \approx 0.37$ compliance, with 6/12 problems strictly mixed — a regime that does not exist for the base model or for any of v9, Phase A, or KTO. *On the clean set, the same checkpoint's compliance drops to* ≈ 0.09. The per-problem flip rate (problems on which the base model is always-loop and the trained model produces ≥ 1 compliant generation) is 19% on the clean set,

$\approx 67\%$ on the val set. Compliance $\wedge$ pass is ≈ 0.04 on the clean set — meaning roughly half of the compliant generations *fail their tests*.

	base	v9	val set (12 prob)		clean set (17 prob)	
		SFT	`-bestval`	`-final`	`-bestval`	`-final`
gens (usable)	$\sim n$=3	71	93	91	70	62
compliance	≈ 0.05	0.07	**0.37**	**0.51**	0.09	**0.26**
compliance $\wedge$ pass	≈ 0.05	0.07	0.34	**0.47**	0.04	**0.16**
flip rate*	–	0/26	8/12	**9/12**	3/16	**6/12**
$[0.3, 0.7]$ zone	0	0	2/12	3/12	2/17	1/12

Table 2: Bare-prompt compliance, AST-rechecked from saved sources. *Per-problem flip rate counts the fraction of always-loop-bare problems for which the trained model produces ≥ 1 compliant generation. The validation set is the 12 LCB problems used for early-stopping (so it is held out from training but not from checkpoint selection); the clean set is 17 further LCB problems not used for either. The clean-set evals had high format-error rates (49% for `-bestval`, 54% for `-final`, `no_code_block_found`, concentrated on harder "D" problems and the `arc` problems) likely due to `max_new_tokens` truncation; val-set rates are $\leq 6\%$ because val problems are shorter. We report compliance on usable generations. Per-problem on the clean set, `-bestval`: `abc385_c`= 1.00, `abc386_c`= 0.38, `abc379_d`= 0.50, `abc359_c`= 1.00 (already bare-compliant for base); other 13 problems at 0.00. Per-problem on the clean set, `-final`: 5 new flips relative to `-bestval` (`abc362_c`= 0.50, `abc363_c`= 0.12, `abc367_d`= 1.00 at noisy n=1, `abc370_c`= 0.12, `abc379_d`= 0.75, `abc386_c`= 0.86); one regression (`abc385_c`: $1.00 \to 0.00$, losing the easy closed-form flip); 5 problems unusable due to all-gen-error truncation. Per-problem on the val set, `-final` vs `-bestval`: 4 strong improvements (`abc356_d`: $0.25 \to 0.88$; `abc377_c`: $0.12 \to 0.62$; `abc384_c`: $0.38 \to 0.62$; `abc367_c`: $0.88 \to 1.00$); 1 new flip (`abc372_c`: $0 \to 0.14$); 1 mild drop (`abc365_d`: $0.57 \to 0.43$); 6 stays (including 2 saturated at 1.00 and 3 all-non-compliant under both).

What is real qualitatively, and what is not. Inspecting clean-set generations: the model *is* trying to avoid loops. Generations on `abc356_c`, `abc362_c`, `abc363_c`, etc. start with `from itertools import product/permutations` or build comprehensions and contain comments like *"we can't use loops, but we can use list comprehensions/max"*. The trained reflex is present and clearly distinct from base behavior. But on these harder problems the model then falls back to a `for` loop somewhere in the body; on the few problems where it does succeed (`abc385_c`, `abc386_c`, `abc379_d`), the result fails tests roughly half the time. The picture is not "SFT did nothing" — it installed a partial, distinguishable behavior — but it is also not "SFT internalized the rule." On the val set the model both *tries* more often and *succeeds* more often, because the val problems are a comprehension-friendly subset (e.g. cuboid-overlap is three fixed-dim max / min comparisons; a DP recurrence on a fixed alphabet folds cleanly into `reduce`).

The selection caveat. The 12 validation problems were used to pick `-bestval` (by NLL) and then reused to report compliance. NLL and AST compliance are correlated only weakly, so the selection pressure is mild — but with $n = 12$ problems and ≈ 4 flips, sampling 5 further problems (the clean set) was enough to bring the apparent generalization down by a factor of ≈ 4. We recommend that any generalization claim for small-data fine-tuning report a clean third set whose problems were used for neither training nor checkpoint selection.

The val-NLL-minimum deploy is not the best checkpoint. We evaluated the over-fit endpoint `-final` (epoch 20, after the val-NLL curve has fully climbed) on the same val and

clean sets, expecting it to confirm the U-shape behaviorally. It did not. On the clean set, `-final` *outperforms* `-bestval` by roughly 3× on raw compliance (0.26 vs 0.09) and ≈ 4× on compliance ∧ pass (0.16 vs 0.04). On the val set — the set `-bestval` was selected to be best on — `-final` also wins, 0.51 vs 0.37 (+36%) on compliance and 0.47 vs 0.34 on compliance ∧ pass. The conditional pass rate among compliant generations is also higher under `-final` on both sets (clean: $10/16 = 62\%$ vs $3/6 = 50\%$; val: $43/46 = 93\%$ vs $32/34 = 94\%$).

The per-problem pattern is structural. On the clean set, `-final` converts 5 previously always-loop hard problems to partial flips (`abc362_c`, `abc363_c`, `abc367_d`, `abc370_c`, `abc379_d`) — the comprehension-unfriendly problems `-bestval` could not crack — and reinforces the partial flips it already had (`abc386_c`: $0.38 \to 0.86$). It loses the one easy full flip `-bestval` had (`abc385_c`: $1.00 \to 0.00$). On the val set, the same pattern: `-final` strongly reinforces partial flips (`abc356_d`: $0.25 \to 0.88$; `abc377_c`: $0.12 \to 0.62$; `abc384_c`: $0.38 \to 0.62$), adds one new partial flip (`abc372_c`: $0 \to 0.14$), and mildly drops one (`abc365_d`: $0.57 \to 0.43$); the saturated 1.00 problems and always-non-compliant 0.00 problems stay where they are. The natural reading is that val-NLL minimum is a poor selector for AST-compliance generalization at this data scale: the over-fit endpoint has installed a more aggressive comprehension-rewrite reflex *without* the loss-in-correctness one would expect from over-fitting, on both the val set used for selection and a clean set neither training nor selection saw.

6 GRPO and a Cautionary Note on Hybrids

We ran GRPO with the verifiable reward $r = \mathbf{1}[\text{compliant} \wedge \text{tests pass}]$ from the base model, $G = 32$, $T = 0.9$, lr $= 2 \times 10^{-6}$, KL anchor with weight 0.02. The run aborted at step 20 with mini-eval pass rate 0. The cause is the bimodality of §4: 14/19 steps had advantage variance ≤ 0 (uniform groups), the remaining steps had advantage variance ≤ 0.06 from very small mixed-group counts, and the policy gradient on those few non-uniform groups drove an off-manifold update that destroyed correctness. The natural next experiment is GRPO from `-bestval` rather than base: the val-set bimodality is partially broken there (6/12 mixed). But the clean-set bimodality is largely intact (≈ 19% flipped, of which only a few are in the discriminating zone), so the available signal from a truly OOD problem distribution is thin, and we have not run that experiment. We note two implementation issues in our GRPO code that any continuation should fix: a KL-reference path that silently fell back to the base policy when the stacked-adapter mode failed, and a raw-summed (rather than mean-normalized) gradient across active rollouts. Both have been corrected in our public code release.

7 Discussion

Three takeaways.

The prompt–default gap on this constraint at this scale is largely an elicitation phenomenon. The constraint is mostly free (Table 1); the bare-prompt non-compliance reflects a per-problem default rather than an inability. This is the structural cousin of password-locked-model elicitation (Greenblatt et al., 2024) — the capability is reliably present under instruction, just inaccessible by default — but on a stylistic AST constraint rather than a quality-locked benchmark.

The per-problem default is near-deterministic and structurally predictable. Bare-prompt compliance happens on problems whose loop-free solution is the natural one (closed-form, builtin-reduce, fixed-dimension max/min); always-loop problems are those whose natural framing is iterative. Training shifts the disposition (`itertools` imports become common, comprehensions are attempted) but does not reliably convert problems whose loop-free path is non-obvious; the bimodality survives most of our training attempts. The implication for verifiable-reward RL is straightforward: GRPO needs within-group reward variance, and most prompt groups don't

have it; the SFT-warmstarted GRPO arm is plausible but not strongly motivated by our SFT result, since most clean-set problems remain bimodal under `-bestval`.

Val-NLL minimum is a poor checkpoint selector for AST compliance at this data scale. The same training run, evaluated at four points (val/clean × `-bestval`/`-final`), tells a consistent story: `-final` beats `-bestval` on *both* problem sets. On the val set used for early-stopping, `-bestval` reaches 0.37 and `-final` reaches 0.51 (+38%). On the clean set, `-bestval` drops to 0.09 (selection inflation) and `-final` reaches 0.26 ($\approx 3\times$). Compliance $\wedge$ pass follows the same pattern. Val-NLL and AST compliance are evidently weakly correlated — weakly enough that the deploy chosen by held-out NLL on a 12-problem val set is materially worse than the same training continued through nominal overfitting, on *both* the set it was selected on and a clean third set. We recommend reporting both deploy and overfit-endpoint numbers, and being skeptical of small-data fine-tunes that report only the val-min deploy.

8 Limitations

One model (Gemma 4 31B-it), one constraint (`no_loops_no_recursion` as operationalized by our AST checks, which permit comprehensions and `reduce`), one benchmark family (LCB-medium plus a HumanEval/MBPP partition for the bimodality measurement), small sample sizes ($n = 8$ on 17 clean problems; effective denominator on the clean-set bestval eval is 70 after 49% format errors). The format-error rate is itself a caveat: a re-run with larger `max_new_tokens` would recover the 4 all-truncation problems and likely tighten the clean-set number, though the qualitative picture would survive. A stricter constraint that disallowed comprehensions and `reduce` would test a more interesting notion of "loop-free," and might break the comprehension-rewrite trick at the heart of the elicitation gap; we have not tried it.

9 Conclusion

We frame bare-prompt non-compliance with a stylistic code constraint as an elicitation gap rather than a cost trade-off. The constraint is mostly free; the bare-prompt distribution is per-problem near-deterministic; the prompt closes most of the gap by triggering a syntactic rewrite the model already knows. A properly-tuned small-data SFT installs a partial reflex — the model imports `itertools` unbidden and attempts comprehensions on held-out problems — but on a truly out-of-distribution problem set it flips $\approx 19\%$ of always-loop problems to some compliance, with the rare compliant generations often producing wrong algorithms. We do not have evidence that further small-data SFT, KTO, or GRPO would meaningfully move the picture; we do have evidence that careful checkpoint reporting matters a lot, and that compliance figures derived from generated source files should be AST-rechecked end-to-end.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023. doi: 10.52202/075280-0441.

Kawin Ethayarajh, Winnie Xu, Niklas Muennighoff, Dan Jurafsky, and Douwe Kiela. KTO: Model alignment as prospect theoretic optimization. *arXiv preprint arXiv:2402.01306*, 2024.

Ryan Greenblatt, Fabien Roger, Dmitrii Krasheninnikov, and David Krueger. Stress-testing capability elicitation with password-locked models. *arXiv preprint arXiv:2405.19550*, 2024.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025.

Emil Ryd, Henning Bartsch, Julian Stastny, Joe Benton, and Vivek Hebbar. Removing sandbagging in LLMs by training with weak supervision. In *International Conference on Machine Learning*, 2026. arXiv:2604.22082.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.